

AI Support Agent for Uber_Support

Hiver SDE Intern Take-Home

Dataset	Customer Support on Twitter (Kaggle, <code>thoughtvector/customer-support-on-twitter</code>)
Brand	Uber_Support, 56,270 brand replies in the full file
Golden set	188 hand-labelled examples, <code>eval/golden_set.jsonl</code>
Retrieval	<code>all-MiniLM-L6-v2</code> (CPU path, used for every number below)
Generation	<code>gemini-3.6-flash</code> for classification, generation and judging

The code auto-selects `Qwen3-Embedding-0.6B` and an optional `Qwen3-Reranker-0.6B` when a GPU is visible, but the committed `index.pkl` behind every result in this report was built on CPU with `all-MiniLM-L6-v2`. Qwen3 numbers are not reported because they were never run end to end.

1 Problem framing

1.1 What “good” means for Uber

Uber support is not e-commerce support with different nouns. Three things shape what a good agent looks like here.

Safety outranks fluency, and it is not close. Around 2% of inbound Uber messages (1,107 of 55,182 by keyword count) reference a collision, harassment, a theft or police involvement. A fluent, polite, wrong reply to any of those is worse than no reply. So the system treats `safety_incident` as a hard stop: always escalated, and unlike every other intent it never gets a draft written at all. Putting a machine-written reply to an assault report in front of a human to approve is still the machine composing that reply.

Twitter is a triage funnel, not a resolution channel. The overwhelming majority of Uber’s own historical replies redirect to DM or the in-app help centre, because fares, trip receipts and driver identity cannot be discussed on a public timeline. So “good” is routing correctly and setting expectations, not resolving the issue in 280 characters.

No inventing policy. The generator is grounded in retrieved past resolutions specifically so it does not improvise refund amounts, ETAs or compensation.

1.2 What was deliberately not built

- **Multi-turn context.** Only the first brand reply in each thread is used. Uber threads redirect to DM almost immediately. This has a real cost, visible in Failure Mode 2.
- **Non-English handling.** A character-script filter drops non-Latin-script messages, mostly Japanese and Hindi. Spanish slips through, which is a known gap rather than a solved problem.
- **Fine-tuning.** Uber’s historical agents are inconsistent. Fine-tuning on their replies would encode that inconsistency as ground truth.
- **Banking77.** Its 77 banking intents do not map onto rideshare traffic, and the brief asks for intents defined from the data.

2 Results against two baselines

All numbers below are computed over the 188-example golden set and regenerate via `python eval/run_eval.py --skip-judge`, which needs no API key. The judge scores in the second table below require API access and are carried from the committed `eval/report.json`.

System	Accuracy	Macro F1	What it is
Baseline 1: keyword match	24.5%	0.236	first substring match wins
Baseline 2: TF-IDF + LogReg	35.1%	0.206	70/30 split, unigrams and bigrams
This system: few-shot LLM	36.2%	0.316	structured JSON, safety pre-filter

2.1 Intent classification

The keyword baseline is deliberately weak and it shows: it cannot place 131 of 188 messages at all and dumps them into `other`. It also cannot tell “the app crashed” from “my driver crashed”, which for a rideshare brand is the difference between a bug report and a collision.

The TF-IDF baseline is the more honest bar. It wins on raw accuracy by learning the head classes and ignoring the tail, which is what its low macro-F1 (0.206) exposes. Accuracy alone would flatter it. The LLM system beats it by 1.1 points of accuracy, which is inside the confidence interval in §4.6 and should not be read as a win.

2.2 Escalation and reply quality

Metric	Result	Reading
Escalation accuracy	59.6%	agreement with human escalate/hold decision
Escalation Cohen’s κ	0.191	the number that actually matters, see §4
Zero-draft safety routing	9	messages that bypassed generation entirely
Reply quality, judge mean	2.99	1 to 5 across four rubric dimensions ($n = 179$)
Judge parse failures	0	malformed JSON from the judge

The escalation split is 80 true positives, 62 false positives, 14 false negatives, 32 true negatives. The system escalates 142 of 188 messages (75.5%) against a gold rate of 50%. It is heavily biased toward escalation, and §3.4 explains why that is mostly an artefact of corpus size rather than a policy choice.

2.3 Is the LLM judge trustworthy

An LLM judge is worth nothing unless it tracks a human. 30 replies were scored independently by hand and compared against the judge.

Agreement metric	Value	Reading
Exact match	66.7%	identical score on 20 of 30
Within 1 point	100.0%	no catastrophic disagreement
Cohen’s κ	0.513	moderate agreement
Judge mean vs human mean	3.27 vs 3.00	the judge runs lenient

The gap between those two means is the useful finding. The judge is systematically kinder than a human, consistently so on vague, non-committal replies. Asked to rate “We’ve followed up via DM!” against a customer who wanted a specific answer, the judge gave 3 and the human gave 2, noting it was evasive. Every reply quality number here should be read with that 0.27 point optimism bias in mind. See §4.7 for a provenance caveat on this table.

3 Failure analysis

Five failure modes, all drawn from the current `eval/predictions.jsonl`. Pair IDs trace back to `eval/golden_set.jsonl`. The system gets 68 of 188 right, so there are 120 errors to account for; the first two modes are 41 of them.

3.1 A phantom class the golden set never contains

`service_outage` is predicted 19 times. The golden set contains it zero times, so all 19 are errors by construction: 11 were `complaint_escalation`, 6 `product_defect`, 1 `billing_refund`, and 1 a genuine safety report.

- 488710_488709: “I do not see available drivers on the map when I open my rider app” → `service_outage`. Gold: `product_defect`.
- 544501_544499: “I have. No reply.” → `service_outage`.

Hypothesis. The taxonomy offers the model a category the evaluation cannot reward, and “the app is not showing me something” reads as an outage to a classifier with no uptime signal to check. The class is real for Uber but unlabelled here, so it functions as a sink. This is a taxonomy-evaluation mismatch, the same class of bug as §4.4, and it was not caught because per-class metrics with empty support get silently zeroed rather than flagged.

3.2 Acknowledgements read as praise

`feedback_praise` is predicted 24 times against 5 gold examples: precision 2/24.

- 128426_128427: “DM done and here also __email__” → `feedback_praise`. Gold: `general_inquiry`.
- 346552_346553: “DM sent with the screenshot of the Kiev trips” → `feedback_praise`. Gold: `general_inquiry`.

Hypothesis. A short, co-operative, non-angry message with no explicit request looks like praise to a classifier that has only the message in front of it. “DM sent” is a mid-thread acknowledgement whose meaning lives entirely in the previous turn, which the first-reply-only decision in §1.2 throws away. This is the concrete cost of that decision, and it is 22 errors rather than the single unclassifiable “Sent” case I originally cited.

3.3 Safety incidents classified as ordinary complaints

Recall on `safety_incident` is 6 of 11. The five misses:

- 997262_997260: “drivers plea for direction, if you don’t hear from me in a few hours, send rescue!!!” → `service_outage`, **not escalated**. The only safety miss that also escaped escalation.
- 1662425_1662424: “i’ve had multiple male drivers record me. is this allowed?” → `complaint_escalation`. Escalated, but with no zero-draft guarantee.
- 2394606_2394605: “One of your cyclists crashed in to my car causing a dent.” → `product_defect`. Escalated.
- 438151_438150: a driver in a physical fight with another driver at the airport → `complaint_escalation`. Escalated.
- 2908499_2909756: a driver’s insurance position after a collision → `complaint_escalation`. Escalated.

Hypothesis. The hard keyword pre-filter fired on only 2 of the 9 `safety_incident` predictions; the LLM produced the other 7. So safety routing is mostly a model judgment, not a rule, and the model reads measured language as measured severity. Covert recording, a fight between third parties, and a cyclist collision are all described calmly and none contain emergency vocabulary. The

category is defined by the nature of the event, not the temperature of the writing, and the prompt does not say so.

3.4 Escalation is driven by corpus size, not risk

Of 62 over-escalations, 51 cite “no closely similar past resolution found” and only 24 cite a high-risk intent. Mean top similarity across the 179 drafted replies is 0.581, and 84 of them fall below the 0.55 grounding threshold.

Hypothesis. The retrieval index holds 608 pairs (§4.2), of which 188 are the evaluation set itself. With a corpus that small, a low similarity score mostly means “nothing like this is in the index”, not “this message is unusual or hard”. The grounding rule is therefore measuring corpus coverage and routing half the traffic to a human on that basis. It is the single largest driver of the 75.5% escalation rate, and it would behave completely differently on a full-size index.

3.5 Residual retrieval leak on duplicated customer tweets

Self-exclusion drops a row’s own `pair_id` before ranking, but 12 of 179 drafts still retrieve a neighbour at similarity ≥ 0.95 . 898285_898287 retrieves 898285_898284: the same customer tweet, paired with a second brand reply.

Hypothesis. A customer tweet that received two brand replies produces two pairs with different `pair_ids` and identical `customer_text`. Excluding by `pair_id` removes one and leaves the other, so the generator can still be shown a human reply to the exact message it is answering. 34 of the 608 indexed pairs share a duplicated `customer_text`. The fix is to exclude on `customer_tweet_id`, or to deduplicate at index build.

4 What is misleading about my headline number

4.1 The evaluation had a retrieval leak, and the first set of numbers was invalid

Golden-set messages also live in the retrieval index. In the first evaluation run, 171 of 188 predictions retrieved the query’s own pair as the *top* hit, 176 retrieved it somewhere in the top three, and 108 recorded a top similarity of exactly 1.0. The generator was being shown the ground-truth reply before writing its own.

Two consequences. Reply quality was inflated, because the model was paraphrasing an answer it had been handed. More subtly, the grounding-similarity branch of the escalation rule never fired once: zero of the drafted rows fell below 0.55. One third of the escalation policy was dead code and the numbers gave no hint of it.

`RetrievalIndex.query` now takes `exclude_pair_id` and the pipeline passes each row’s own ID. The affected results are archived unused in `eval/stale_pre_fix/`. The leak is reduced but not eliminated, as Failure Mode 5 shows. The general lesson: a similarity of exactly 1.0 on more than half the rows should have been read as a bug immediately rather than as strong retrieval.

4.2 The grounding corpus is 608 pairs, not 56,270

The brand has 56,270 replies in the raw file, and that number appears at the top of this report because it is what motivated the brand choice. It is not what the system runs on. The subsample keeps every Uber row but only 30,000 random non-Uber rows, so most of the customer tweets those replies point at are absent and the join drops them: 608 pairs survive. 188 of those are the golden set, leaving roughly 420 genuine grounding examples.

Every retrieval and grounding number in this report is a property of a 608-pair index. The escalation rate in particular (Failure Mode 4) would move substantially on a full index. This is the single biggest gap between what the report appears to measure and what it measures.

4.3 Escalation accuracy is the wrong number to quote

The golden set splits 94 to 94 on escalate versus hold. On a perfectly balanced set a coin flip scores 50% and a system that escalates everything scores 50%, so raw accuracy has a floor that looks respectable while meaning nothing. Cohen’s κ is reported alongside it for that reason, and $\kappa = 0.191$ is the number to argue about.

The balance is itself an artefact of stratified sampling. Real Uber traffic is nowhere near 50% escalation-worthy, so neither figure transfers to production volumes without recalibration.

4.4 A taxonomy change silently broke an earlier set of intent numbers

Partway through, an extra intent (`feedback_negative`, splitting venting from actionable complaints) was added to the classifier. The golden set had already been labelled without it. The system then predicted a label the ground truth could not contain 55 times. Because gold can never carry that label, all 55 were errors, out of 138 total, and accuracy sat at 0.266.

Nothing in the evaluation flagged this. It looked like a bad model. The intent was removed to match what the golden set encodes. Note that `service_outage` is the same bug still live, in the other direction: a class the taxonomy has and the labels do not.

4.5 The TF-IDF baseline is trained on 131 examples

Nine classes present, 131 training rows, roughly 15 per class, with three classes in single digits (`safety_incident` 7, `account_access` 7, `feedback_praise` 3). It is a directional sanity check and nothing more. It should not be read as a ceiling on what classical methods could do here.

4.6 $N = 188$ is small, and the interval is wide

A 95% Wilson interval on a proportion near 0.40 at $N = 188$ spans roughly ± 7 points (half-width 6.9). The 1.1-point gap between this system and the TF-IDF baseline is well inside that and is not evidence of anything. `service_outage` drew zero labels, so its per-class recall is undefined rather than zero, and macro-F1 averages over nine classes, not ten.

4.7 The judge-human agreement table is not traceable to this prediction run

`eval/human_judge_check.jsonl` holds 30 hand-scored replies, but none of them match the replies in the committed `eval/predictions.jsonl`, and one scores Uber’s own historical reply rather than a generated one. The $\kappa = 0.513$ figure therefore validates the judge against a run that is not the one reported here. The judge may well be equally reliable on this run, but that has not been demonstrated, and every reply-quality number in §2.2 inherits that uncertainty.

4.8 Nothing here measures faithfulness

The judge rates whether a reply reads as correct. It does not check the reply against the retrieved context. A reply that contradicts the historical resolution it was grounded on can still score well by sounding right, and Failure Mode 5 is exactly the setup in which that would go unnoticed.

5 What one more week would go on

1. **Reconcile the taxonomy with the labels.** Either label `service_outage` examples into the golden set or drop the class from the prompt. 19 of 120 errors are currently unwinnable by construction. Same discipline that fixed `feedback_negative`.

2. **Fix safety recall.** 6 of 11 is not shippable, and the pre-filter is carrying only 2 of 9 safety routings. Add few-shot examples covering calm harassment, covert recording, third-party collisions and theft, and evaluate safety recall as a standalone metric with its own threshold rather than letting 11 examples sit inside macro-F1.
3. **Rebuild the index properly.** Pull the parent customer tweets of every brand reply instead of relying on a random subsample to contain them, and deduplicate on `customer_tweet_id`. That fixes Failure Mode 5 and makes the grounding threshold in Failure Mode 4 mean something. Only then are the escalation thresholds (0.75 and 0.55, never tuned) worth sweeping.
4. **Two turns of thread context.** Enough to resolve “DM sent” and “Sent”, which is 22 errors in Failure Mode 2. Full dialogue state tracking is not needed.
5. **Re-score 30 replies from this run** so the judge validation describes the system being reported, and add a faithfulness dimension that checks the draft against its retrieved context.

Reproduction: `README.md`. Decisions and reasoning: `decision_log.md`. Golden set sampling and labelling: `eval/golden_set_schema.md`. Every figure in §2 regenerates via `python eval/run_eval.py -skip-judge` and `python scripts/report_numbers.py`, both of which run without an API key.